

EffectiveGPU

技术白皮书

顺丰科技团队
2025年4月



目录

CATALOGUE

概述	01
背景介绍	02
2.1 GPU 算力发展的挑战	02
2.2 GPU 池化和虚拟化的价值	02
2.3 EffectiveGPU 的技术目标	03
技术原理与架构	05
3.1 技术架构	05
3.2 核心组件	06
3.3 Volcano 集成方案	07
3.4 HAMi 兼容方案	07
3.5 GPU 虚拟化实现原理	08
关键创新点	09
4.1 创新点一：针对多节点异构GPU的池化和调度实现	09
4.2 创新点二：抽象适配国产和海外GPU的统一调度接口	10
4.3 创新点三：实现GPU的显存和算力切分保障机制	11
4.4 创新点四：通过显存超分和优先级保证多任务并行运行	12

实施部署与测试	13
5.1 云原生部署	13
5.2 基于 Volcano 调度器部署	14
5.3 部署功能测试	15
5.3.1 使用整卡 EGPU	15
5.3.2 算力和显存切分	15
5.3.3 配置任务优先级	16
应用场景与解决方案	17
6.1 大模型推理服务场景	17
6.2 测试服务集群场景	17
6.3 语音识别场景	17
6.4 适配国产算力的推理场景	17
结论	18
附录：名词解释与参考资料	19

1.概述

在AI大模型推动产业智能化变革的新时代，算力资源的高效配置已成为制约企业数字化转型的核心命题。当前大模型推理服务呈现指数级增长态势，异构算力管理效率不足已成为掣肘企业AI应用落地、抬高运营成本的关键瓶颈。传统的异构算力资源分配模式（如独占 GPU）分配模式暴露三大核心痛点：**资源孤岛导致跨节点算力无法动态复用、粗粒度调度引发的资源浪费、以及异构环境适配性不足造成的管理复杂度攀升。**

针对行业痛点，本白皮书介绍了 EffectiveGPU 池化技术（简称 egpu），通过统一调度接口标准、算力细粒度切分与跨节点协同调度，可**显著提升集群 GPU 等异构算力的利用率和管理效率**，为云端及边缘场景提供更灵活、更高效的算力基础设施，并且**适配国产 AI 算力平台**。作为构建 GPU 池化和虚拟化的算力基础设施核心技术，深入融合自研的 AI 技术平台，为**构建自主可控的智能计算体系**提供关键技术支撑。

2. 背景介绍

2.1 GPU 算力发展的挑战

随着大模型技术与AI应用的爆发式增长，算力资源的高效利用成为产业核心诉求。传统GPU资源分配模式存在**利用率低**（平均<30%）、**弹性不足**（整卡独占）、**生态碎片化**（多厂商硬件/协议差异）等诸多痛点：

算力资源利用率低

在AI大模型运行环境中，常规的GPU分配机制多采用独占模式，导致计算资源空置问题突出。特别是在模型推理和测试验证环节，GPU设备的算力负荷率和显存使用率普遍处于低下状态。

资源共享力度不足

现有GPU资源调度方案通常局限于整卡分配，缺乏灵活的计算单元与显存空间切分机制。这种粗放式资源分配方式难以支撑多样化AI任务在单张加速卡上的并行执行需求。

异构硬件适配困难

当前加速器市场呈现多元硬件生态（涵盖GPU/NPU/各类自研芯片），不同厂商设备存在兼容壁垒，导致上层应用面临多平台适配成本高企的挑战。

2.2 GPU 池化和虚拟化的价值

基于创新的池化技术架构，实现**异构算力细粒度切分**、**统一的调度接口与云原生支持**，显著提升算力资源利用率（最高达200%显存超分能力），为AI训练、推理及科学计算场景提供灵活高效的算力基础设施支撑：

资源使用效率优化

采用精细化资源分割与协同管理机制，突破传统单卡独占模式，有效激活闲置GPU算力的动态复用能力。

统一调度的接口标准

计算框架服务提供层以调度器插件（vGPU scheduler-plugin）以及定制化的异构算力设备插件（device-plugin）的形式，实现整个集群对于异构算力资源的复用请求。

增强云原生支持

深度融合Kubernetes原生插件架构，强化容器化环境对GPU等异构计算资源的全生命周期管控能力。

2.3 EffectiveGPU 的技术目标

EffectiveGPU 项目的主要目的是提供一个异构算力虚拟化平台，用于管理和优化大规模异构算力集群中的资源利用，在云原生环境中简化部署和利用如GPU这样的复杂硬件的过程，同时提高资源的利用效率和灵活性。

具体来说，EffectiveGPU 项目旨在实现以下几个关键技术目标：

- 1. 异构设备管理：**EffectiveGPU具备多样化异构设备（包括GPU、NPU等）的统一管理能力，支持跨Pod的设备共享机制，通过分析硬件拓扑结构特征并应用智能调度策略，实现更优的资源分配决策。
- 2. 设备共享与资源隔离：**系统提供细粒度资源管控方案，支持按核心利用率（百分比）和显存容量（MB）进行精确分配，对计算单元实施硬件级隔离。在保持业务无须改造的前提下，实现资源分割带来的性能损耗控制在5%以内。

- 3. 弹性资源超配：**支持GPU算力与显存的双维度超分技术，通过动态算力复用机制实现空闲资源跨应用调度，配合优先级队列保障高优先级任务QoS。基于统一内存架构实现200%显存超分比，突破单卡物理显存限制，支持多任务并发执行。
- 4. 资源效率优化：**采用创新虚拟化技术构建高密度资源复用体系，通过精细化资源调度策略提升硬件使用效率，有效降低闲置资源浪费，实现设备利用率的质的飞跃。
- 5. 无缝兼容适配：**采用非侵入式设计架构，确保现有业务系统无需任何改造即可平滑接入，全面兼容各类存量应用程序的运行环境。
- 6. 智能调度体系：**内置多维度调度策略引擎，支持基于节点特征、GPU型号等参数的自适应调度算法，持续优化集群资源分配效率。
- 7. 精准设备调度：**提供基于设备型号指纹和唯一标识符的精准调度能力，确保业务负载与硬件特性实现最优匹配。

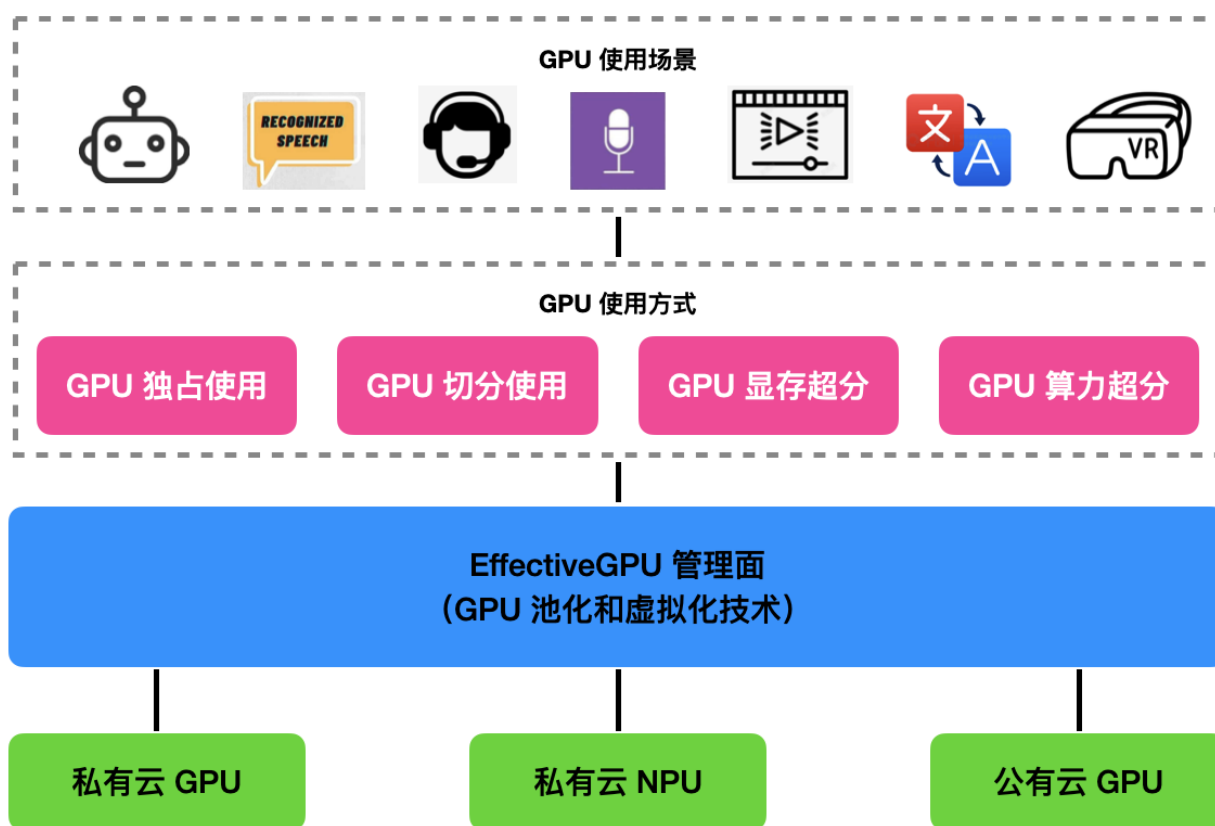
EffectiveGPU 作为自主研发的方案，在多方面展现显著优势。其**免费特性**，相比其他商业方案的付费模式，可节省软件授权成本。功能上，**支持显存切分、算力切分以及支持显存超分和算力超分**，突破硬件限制，提升资源利用率。同时**支持计算优先级、训练和推理混部**，满足灵活调度与高效部署需求。添加池化层后**最低仅下降0.5%性能**，最大程度保证 GPU 性能。提供**运维与监控方面提供支持**，且在公有云场景下**无节点数限制**，适用范围更广。

3. 技术原理与架构

EffectiveGPU 提出了一套支持算力细粒度划分、节点内算力调度、算力跨节点协同调度的 GPU 池化和虚拟化技术架构，通过 GPU 池化和混合云技术方案，实现所有 GPU 资源的统一视图，按需申请和弹性扩缩容。

3.1 技术架构

EffectiveGPU 整体架构图如下：



EffectiveGPU 以 GPU 池化和虚拟化技术为核心，通过管理面实现对 GPU 资源的高效整合与调配。可整合私有云和公有云的 GPU 和 NPU 资源，支持 GPU 独占、切分、显存超分和算力超分等多种使用方式，满足机器人、语音识别、视频处理、翻译和 VR 等多样化应用场景，提升 GPU 资源的利用率和灵活性。

3.2 核心组件

EffectiveGPU 包含一个核心关键组件，与调度监控相关包含额外的多个核心组件。

核心组件	功能描述	应用场景
egpu-core	劫持 CUDA Runtime(libcudart.so) 和 CUDA Driver(libcuda.so) 之间的 API 调用来实现切分超分等功能	实现 CUDA 的显存和算力切分，超分，优先级调度等功能
egpu-device-plugin	用于发现 GPU,上报 GPU 数量，替换 nvidia-device-plugin，同时实现 Volcano 集成逻辑，实现资源注册，资源管理，环境注入等功能	异构设备抽象层，统一管理 NVIDIA/昇腾/寒武纪等厂商硬件资源
egpu-scheduler	用于将申请的egpu调度到egpu节点上	支持Best-Fit/Bin-Packing等策略，动态优化资源分配
egpu-webhook	如果申请的资源为egpu资源，注入egpu-scheduler	将申请egpu的pod交由egpu-scheduler调度器接管调度
effective-gpu-webui	提供Web 界面来管理和扩展 egpu 的功能，实现egpu的监控	可视化和管理节点上的 egpu 资源分配和使用情况。支持任务级别和显卡级别的详细视图

3.3 Volcano 集成方案

为了进一步与 Volcano 集成，本方案还提出了新的技术改造项，增强 Volcano vgpu 使用特性。

1. **新增显存超分能力**，通过 device-memory-scaling 参数动态调配显存超分系数，支持按不同的节点配置不同超分系数，同时保留原有的切分逻辑。
2. **通过 Volcano 实现 NUMA 亲和调度，提升 GPU 资源使用效率**，NUMA 节点是构成非统一内存访问架构（Non-Uniform Memory Access, NUMA）系统的基本单位，一个 NUMA 集合是指单个 Node 节点上多个 NUMA 节点的组合，用于实现计算资源的有效分配，并降低处理器间内存访问的竞争。当 CPU 没有绑核或没有与 GPU 分配在相同 NUMA 节点上时，可能会由于 CPU 争抢或 CPU 与 GPU 跨 NUMA 通信导致应用执行性能下降。为了使应用的执行性能最大化，可以选择将 CPU 与 GPU 绑定在相同 NUMA 节点下。
3. **通过 Volcano 与资源池特性实现并支持 EffectiveGPU 优先级 QoS，混部等特性。**

3.4 HAMi 兼容方案

HAMi 作为目前主流的开源异构 AI 计算虚拟化中间件，提供了丰富的异构 GPU 管理接口，EffectiveGPU 采用了兼容 HAMi 生态的架构实现方案，其核心兼容性设计体现在以下三个维度：

虚拟化接口兼容

虚拟化实现上兼容 HAMi-core 的使用方式，通过覆盖主流 CUDA 版本的接口虚拟化技术，实现原生 CUDA 应用的无缝接入，支持上下文隔离、时间切片等核心特性，确保不同任务间的算力隔离与资源分配时序一致性，使现有 GPU 应用无需改造即可透明化运行。

调度接口兼容

在调度和监控层面深度对齐 HAMi 的云原生调度器功能，支持主流 Kubernetes 集群版本和 Volcano 调度器的深度集成，实现从资源请求、配额分配到任务编排的全链路标准化对接，确保现有调度策略与业务工作流无需重构即可平滑迁移至虚拟化环境。

异构 GPU 兼容

通过集成 HAMi 生态提供的国产算力 device plugin，实现基于统一的抽象驱动框架和跨架构算力调度模型，实现对多种硬件加速卡的混合纳管与协同调度，结合拓扑感知优化能力，支持异构集群的资源池化与任务智能编排，实现不同硬件生态的高效整合。

3.5 GPU 虚拟化实现原理

技术维度	实现方式	性能指标
显存隔离	虚拟内存映射+页表隔离，支持动态分配（1MB~整卡）	保证显存独占避免 OOM，性能损耗 <5%
优先级调度	QoS策略引擎，实时抢占低优先级任务资源保障高优先级服务	推理任务延迟波动 <10ms
算力复用	基于时间片的调度机制，通过任务队列管理和调度不同任务的执行时间，支持多任务并发	资源利用率提升 30% 以上，任务响应时间 <20ms
显存超分	基于 CUDA 的 Unified Memory 实现显存的动态扩展和优化，支持设备显存和内存的共享和管理	显存扩展比例可达 200%，数据传输延迟 <2ms
应用无感	通过系统调用 Hook 实现，无需修改应用代码逻辑，自动适配底层资源变化。	CUDA 应用兼容性达到 100%

4. 关键创新点

4.1 创新点一：针对多节点异构GPU的池化和调度实现

算力池化与调度

提出了一套支持算力细粒度划分、节点内算力调度、算力跨节点协同调度的国产化深度学习计算框架，从AI应用层、AI框架层、服务层、系统软件层和硬件核心层实现了异构国产AI算力平台的池化及调度。

算力细粒度切分

通过API拦截实现了算力细粒度切分，使任务能按需使用算力资源，避免资源浪费。

资源绑定与调度

通过服务提供层为AI应用绑定资源池中的资源，实现了国产化AI算力的调度框架。

技术实现细节

服务提供层和系统软件层面分别使用云原生调度工具（kubernetes）与容器工具（docker）实现。服务提供层实现异构算力的任务分发和资源池的组建；系统软件层利用替换任务调用链中的 `cudaDriver` 相关函数以及利用设备提供商提供的设备复用工具来确保任务使用的资源符合规定与限制。

调度策略研究

针对不同GPU算力的调度策略研究，形成了一套完善的调度机制，突破了 Kubernetes 集群中异构算力独占使用的限制，提升了集群利用率。

4.2 创新点二：抽象适配国产和海外GPU的统一调度接口

■ 插件形式实现资源复用

计算框架服务提供层以调度器插件(vGPU scheduler-plugin)以及定制化的异构算例设备插件(device-plugin)的形式,实现整个集群对于异构算力资源的复用请求。

■ 任务资源声明与调度

每个任务提交到 Kubernetes 集群中时需声明其索要使用的异构算力类型及其规格,调度器插件查询任务信息后负责调度工作,普通任务交由kubernetes自带的默认调度器处理。

■ 调度策略与资源分配

调度器插件中以无状态形式保存任务的异构算力资源信息,支持多种调度策略,根据策略将节点分配到合适节点,并将资源信息注入任务。

■ 设备插件与任务启动

异构算力设备插件识别任务备注信息,设置环境变量、映射驱动文件和设备,交由容器工具启动任务。

■ 资源限制层实现

系统软件层为容器内的资源限制层,能限制容器内对异构设备相关资源的使用。通过研究不同厂商算力接口,构建基于异构算力调度需求的统一接口标准,屏蔽厂商接口差异,实现异构算力调度的统一。

4.3 创新点三：实现GPU的显存和算力切分保障机制

国产AI算力支持与适配

计算框架支持华为Ascend、百度昆仑、算能智能卡等国产AI算力，不依赖国外技术，同时适配主流AI框架和国产AI框架，如Paddle。

细粒度双端算力切分

提供细粒度的AI算力切分方式，解决云平台中算力资源浪费和无法共享的问题，通过API拦截实现切分，具有高自由度、强灵活性和高可扩展性。

跨节点算力协同调度

研究AI算力协同调度方法，充分利用低算力节点资源，缓解高算力节点资源竞争，实现跨节点算力调度，提升云AI算力平台的效率、可用性和降低资源竞争。

自适应国产化AI算力调度

针对国产化AI算力资源，研究适配的调度算法，根据任务类别和资源亲和性，通过预测算法确定分配的算力种类及份额，实现精准调度。考虑节点间异构资源拓扑信息，执行最小冲突的资源分配方案，支持弹性调度策略。

不同设备资源限制方式

针对不同设备，采用相应的方式进行资源限制。如GPU任务通过预加载GPU控制库实现资源审计；MLU任务使用寒武纪提供的容器复用工具；DCU任务与MLU类似；NPU任务通过设置中间层限制资源；算能智算卡和昆仑芯则分别利用其提供的开发工具包和运行时环境实现资源限制。

解决技术难点

解决了云原生场景下异构设备复用的诸多限制、业务中资源限制问题、GPU复用中的兼容性问题、显存统计问题以及不同厂商复用机制冲突等问题。

4.4 创新点四：通过显存超分和优先级保证多任务并行运行

显存超分技术

引入显存超分技术，在物理显存不变的情况下，通过优化显存分配策略和管理机制，使系统能够支持更多的任务同时运行，有效解决多任务对显存资源的高需求问题，提高显存的利用率和系统的并行处理能力。

任务优先级划分

根据任务的紧急程度、资源需求和业务重要性等因素，对任务进行优先级划分。高优先级任务在资源分配和调度中享有更高的权重，确保关键任务能够及时获得足够的资源支持，优先执行和完成，提升系统对重要业务的响应速度和服务质量。

资源分配与调度机制

设计了一套灵活的资源分配与调度机制，根据任务的优先级和资源需求，动态地为其分配合适的显存和其他计算资源。在资源紧张的情况下，优先保障高优先级任务的资源供应，同时合理调整低优先级任务的资源使用，确保系统资源得到最优利用。

资源抢占与回收策略

制定了资源抢占与回收策略，允许高优先级任务在需要时抢占低优先级任务的部分资源，以满足自身的需求。同时，当任务完成或资源使用完毕后，及时回收资源，使其能够被其他任务重新利用，提高资源的周转效率。

性能优化与负载均衡

通过性能优化手段，减少任务在等待资源时的延迟和阻塞，提高任务的执行效率。同时，考虑系统的整体负载情况，实现负载均衡，避免某些资源被过度占用而导致系统性能下降，确保多任务并行运行时系统的稳定性和高效性。

5. 实施部署与测试

5.1 云原生部署

设置部署 effective-scheduler 节点的 label。

```
kubectl label node [nodename] kubernetes.aiplat.resourcePool=aiplat-sys
```

设置部署 effective-device-plugin 节点的 label。

```
kubectl label node [nodename] kubernetes.io/gpu=egpu
```

通过 Helm 部署 effective-gpu 组件。

```
helm install --create-namespace effective-gpu -n effective-gpu ./charts/hami
```

运行 E2E 测试。

```
E2E_TYPE=pullrequest KUBE_CONF=~/.kube/config make e2e-test
```

5.2 基于 Volcano 调度器部署

设置部署 effective-device-plugin 节点的 label。

```
kubectl label node [nodename] kubernetes.io/gpu=egpu
```

部署 Volcano 组件。

```
kubectl apply -f ./volcano-egpu-device-plugin.yml
```

Volcano 配置调整。

```
kubectl edit cm -n volcano-system volcano-scheduler-configmap
-----
kind: ConfigMap
apiVersion: v1
metadata:
  name: volcano-scheduler-configmap
  namespace: volcano-system
data:
  volcano-scheduler.conf: |
    actions: "enqueue, allocate, backfill"
    tiers:
      - plugins:
          - name: priority
          - name: gang
        - plugins:
            - name: drf
            - name: deviceshare
              arguments:
                deviceshare.VGPUEnable: true
            - name: predicates
            - name: proportion
            - name: nodeorder
            - name: binpack
```

5.3 部署功能测试

5.3.1 使用整卡 EGPU

指定使用1张 egpu，显存和算力为默认（即：100% 显存，100% 算力）。

- `nvidia.com/egpu: 1`

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
  namespace: effective-gpu
spec:
  containers:
  - name: main
    image: gpu-cuda-test:v1
    command: ["bash", "-c", "sleep 24h"]
    resources:
      limits:
        nvidia.com/egpu: 1
  tolerations:
  - operator: Exists
```

5.3.2 算力和显存切分

指定算力比例和显存大小，使用2张 egpu，每张 GPU 卡显存为 3000MB 和算力为 30%。

- `nvidia.com/egpu: 2`
- `nvidia.com/egpumem: 3000`
- `nvidia.com/egpucore: 30`

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
  namespace: effective-gpu
```

```
spec:
  containers:
    - name: main
      image: gpu-cuda-test:v1
      command: ["bash", "-c", "sleep 24h"]
      resources:
        limits:
          nvidia.com/egpu: 2
          nvidia.com/egpumem: 3000
          nvidia.com/egpucores: 30
  tolerations:
    - operator: Exists
```

5.3.3 配置任务优先级

配置 pod 任务优先级，0为高优先级，1为低优先级，当出现算力竞争时，优先将算力分配给高优先级任务。

- nvidia.com/priority: 0

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
  namespace: effective-gpu
spec:
  containers:
    - name: main
      image: gpu-cuda-test:v1
      command: ["bash", "-c", "sleep 24h"]
      resources:
        limits:
          nvidia.com/egpu: 2
          nvidia.com/gpumem-percentage: 30
          nvidia.com/egpucores: 30
          nvidia.com/priority: 0
  tolerations:
    - operator: Exists
```

6. 应用场景与解决方案

6.1 大模型推理服务场景

在 AI 生产模型服务中，传统的 GPU 资源分配方式往往导致资源利用率低和成本高。通过采用 EffectiveGPU 技术，可以显著提升资源利用率并降低运营成本。例如，已切换 EffectiveGPU 的 AI 生产模型服务，使用28张 GPU 卡部署65个服务，节省了37张卡。这种方式不仅提高了 GPU 的利用率，还使得服务部署更加灵活，能够根据实际需求动态调整资源分配，避免了资源的浪费。

6.2 测试服务集群场景

在集群测试服务中，资源的灵活调配和高效利用对于测试效率和成本控制至关重要。EffectiveGPU 技术通过算力和显存的切分，使得测试服务能够根据不同的测试任务需求，灵活地分配 GPU 资源。例如，已切换 EffectiveGPU 的集群测试服务，使用6张测试用的 GPU 卡部署19个服务，节省了13张卡。这不仅提高了测试效率，还降低了测试成本，使得资源能够得到更合理的利用。

6.3 语音识别场景

语音识别服务需要高效的算力支持来保证实时性和准确性。EffectiveGPU 通过优先级调度和资源超配，为语音识别提供了灵活的算力支持。可以根据语音识别任务的紧急程度和资源需求，动态地分配 GPU 资源，确保高优先级任务能够及时获得足够的资源支持，提高语音识别的服务质量。

6.4 适配国产算力的推理场景

EffectiveGPU 技术不仅支持主流的 GPU 硬件，还适配了华为昇腾、百度昆仑等国产 AI 算力平台。这为国产 AI 技术的应用和推广提供了有力的支持，使得在国产化背景下，也能够实现高效的算力管理和资源利用。例如，在使用国产 AI 芯片的场景中，通过 EffectiveGPU 的调度和管理，可以充分发挥国产 AI 算力的优势，推动国产 AI 技术的发展和应用。

7. 结论

GPU 池化技术针对云原生环境下异构算力的低利用率、跨节点共享不足以及调度灵活性受限等痛点，提供了完整的解决方案。通过统一调度接口、细粒度算力切分与跨节点资源协同，可**大幅提升集群算力使用效率、降低 TCO 并提升业务部署的灵活性**。

本项目将持续深化对 GPU 池化和虚拟化技术的研究和实践，该技术已在中国某头部快递物流企业的 AI 平台等场景验证，未来将持续推动异构算力生态融合，不断完善云原生 AI 基础设施的技术体系与生态，助力数字经济发展。

附录：名词解释与参考资料

1. Kubernetes 官方文档

<https://kubernetes.io/>

2. Nvidia GPU vGPU 与 MPS

· vGPU: <https://docs.nvidia.com/grid/>

· MPS: <https://docs.nvidia.com/deploy/mps/index.html>

· UnifiedMemory: <https://developer.nvidia.com/blog/unified-memory-cuda-beginners/>

3. 百度昆仑芯

<https://kunlun.ai/>

4. HAMi 开源项目

<https://github.com/Project-HAMi/HAMi>

5. Volcano 开源项目

<https://github.com/volcano-sh/volcano>

致谢

在 EffectiveGPU 池化技术研发过程和落地中，我们得到了众多合作团队的慷慨支持，他们为我们提供了项目所需的硬件设备、专业的技术支持以及稳定的测试环境，在此向他们致以诚挚的感谢。同时，也要对团队研发人员表示感谢，他们在项目中积极地探索创新，为项目贡献出了自己的智慧与力量。

期待 EffectiveGPU 池化技术能够在更广泛的云原生场景中得到应用，为行业带来新的创新与价值，为产业数智化转型添砖加瓦，共同推动技术的进步与发展。

